Name - Chaitanya Kelkar

Roll no. - 53

Experiment 7

Aim - Build an Artificial Neural Network (ANN) using Keras/TensorFlow

2. Dataset Source & Description

- **Source:** Kaggle (Crop_recommendation.csv)
- **Description:** Multivariate agricultural dataset containing soil and weather attributes.
- **Size:** 2200 instances, 8 columns.
- **Target Variable:** label (22 distinct crop categories).
- **Features:** N, P, K (Nutrients) and temperature, humidity, ph, rainfall (Environment).

3. Mathematical Formulation of the Algorithm

An Artificial Neural Network operates through a sequence of forward propagation and backpropagation. During forward propagation, the network processes input data through hidden layers of interconnected nodes (neurons). Each connection has a weight, and each node has a bias. The mathematical operation at a specific node calculates the weighted sum of inputs plus the bias, represented as:

$z = \Sigma (w_i * x_i) + b$

This linear output is then passed through a non-linear activation function, such as ReLU (Rectified Linear Unit), which outputs the input directly if it is positive, and zero otherwise, allowing the network to learn complex patterns.

For multi-class classification, the final output layer uses the Softmax activation function. Softmax converts the raw output scores of the network into a normalized probability distribution across all 22 crop classes, where the sum of all probabilities equals 1. The formula for Softmax is:

$P(y=j \mid x) = e^{(z_j)} / \Sigma e^{(z_k)}$

Once the predictions are made, the network calculates its error using a loss function like Categorical Crossentropy. During backpropagation, the algorithm uses optimization techniques (like Adam or Gradient Descent) to compute the gradient of the loss function with respect to every weight, updating the weights iteratively to minimize the error over multiple passes (epochs).

4. Algorithm Limitations

- **Computational Cost:** ANNs require significant processing power and time, especially as the number of layers (depth) and neurons increases.
- **Black Box Nature:** Unlike Linear SVM or Decision Trees, it is extremely difficult to extract exact "Feature Importance" from an ANN. The internal mathematical weights are distributed across hundreds of nodes, making it hard to explain exactly *why* the model chose a specific crop.
- **Prone to Overfitting:** Neural networks are powerful memorizers. If not controlled properly (using techniques like Dropout or Early Stopping), they will memorize the training data and fail on new, unseen data.

5. Methodology / Workflow

The experiment followed an end-to-end Deep Learning classification pipeline:

- **Data Preprocessing:** Separated the 7 numerical features and the target label. Applied StandardScaler to the features because Neural Networks are highly sensitive to unscaled data (it destabilizes the gradient updates).
- **Target Encoding:** Converted the 22 string labels (e.g., 'rice', 'maize') into numerical integers using LabelEncoder, which is required for Keras.
- **Model Architecture:** Built a Sequential Keras model with an input layer, two hidden Dense layers utilizing 'ReLU' activation, and a final output Dense layer of 22 nodes utilizing 'Softmax' activation.
- **Compilation:** Compiled the model using the 'Adam' optimizer and 'sparse_categorical_crossentropy' as the loss function (ideal for integer-encoded multi-class targets).
- **Training:** Trained the model for 50 epochs with a batch size of 32, holding out a validation set to monitor for overfitting during training.

6. Performance Analysis

The model's effectiveness was evaluated primarily through Learning Curves and a Confusion Matrix. By plotting the Accuracy and Loss curves across all 50 epochs, we can verify that the network converged properly—the training loss consistently decreased while the validation accuracy stabilized at a high percentage without diverging. The classification report and confusion matrix confirmed high precision and recall across all 22 crop classes, demonstrating that the network's hidden layers successfully learned the non-linear boundaries between complex environmental profiles.

7. Hyperparameter Tuning

In deep learning, architecture and training parameters must be carefully tuned:

- **Epochs & Batch Size:** Epochs dictate how many times the network sees the entire dataset. Batch size dictates how many samples it processes before updating its weights.
- **Hidden Layers & Neurons:** Increasing the number of layers allows the network to learn more complex features, but requires more data to train effectively.

- **Optimizer Learning Rate:** Controls how drastically the weights are updated during backpropagation. The Adam optimizer automatically adapts this rate dynamically.

Code -

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix



# Import TensorFlow & Keras

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout



# 1. DATA LOADING

print("Loading Data...")

try:

    df = pd.read_csv('./data/Crop_recommendation.csv')

    print("Data loaded successfully.")

except FileNotFoundError:

    print("Error: File not found. Check your data path.")
```

```
    exit()


# 2. DATA PREPROCESSING

X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

y = df['label']


# Neural Networks require numerical targets. Convert crop names to
integers (0 to 21)

label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)


# Split data (80% Train, 20% Test)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42, stratify=y_encoded)


# Feature Scaling is MANDATORY for Neural Networks

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# 3. BUILDING THE ARTIFICIAL NEURAL NETWORK

print("\n--- Building the ANN Model ---")


model = Sequential()
```

```
# Input Layer & First Hidden Layer
# We have 7 features, so input_shape=(7,)
# We use 64 neurons and ReLU activation
model.add(Dense(units=64, activation='relu', input_shape=(7,)))

# Second Hidden Layer
model.add(Dense(units=32, activation='relu'))

# Output Layer
# We have 22 crops, so we need exactly 22 neurons
# We use Softmax activation for multi-class probabilities
model.add(Dense(units=22, activation='softmax'))

# Compile the model
# 'sparse_categorical_crossentropy' is used when labels are integers
rather than one-hot encoded
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model.summary()

# 4. TRAINING THE MODEL
print("\n--- Training the ANN ---")
```

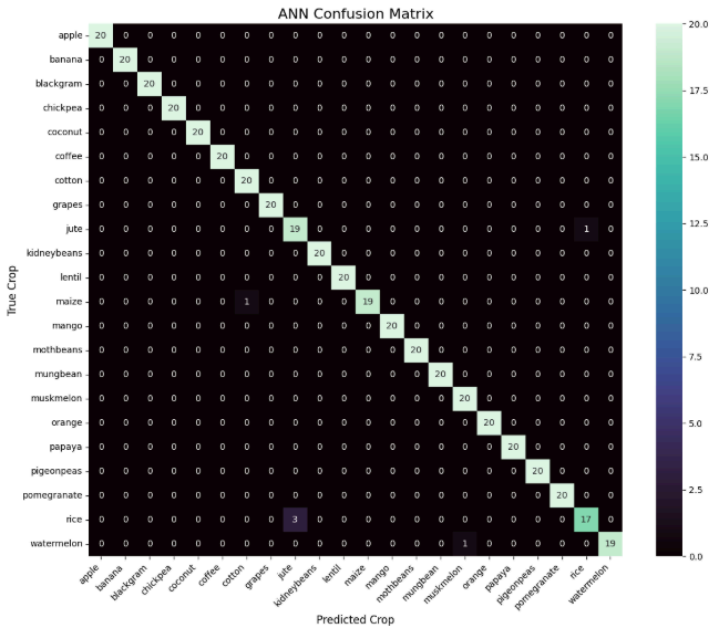
```
# Train for 50 epochs, update weights after every 32 samples

# validation_split=0.2 takes 20% of the training data to monitor
overfitting internally

history = model.fit(X_train_scaled, y_train,

                    epochs=50,

                    batch_size=32,

                    validation_split=0.2,

                    verbose=1)



# 5. EVALUATING & PLOTTING LEARNING CURVES

print("\n--- Evaluating Model Performance ---")



# Evaluate on the unseen Test Set

test_loss, test_accuracy = model.evaluate(X_test_scaled, y_test,
verbose=0)

print(f"Final Test Accuracy: {test_accuracy*100:.2f}%")



# Plotting Accuracy and Loss Curves

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))



# Accuracy Curve

ax1.plot(history.history['accuracy'], label='Train Accuracy',
color='blue', linewidth=2)

ax1.plot(history.history['val_accuracy'], label='Validation Accuracy',
color='orange', linewidth=2)
```

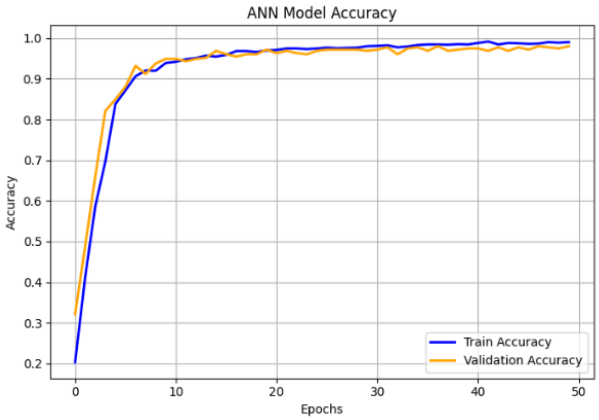
```
ax1.set_title('ANN Model Accuracy')

ax1.set_xlabel('Epochs')

ax1.set_ylabel('Accuracy')

ax1.legend()

ax1.grid(True)



# Loss Curve

ax2.plot(history.history['loss'], label='Train Loss', color='blue',
linewidth=2)

ax2.plot(history.history['val_loss'], label='Validation Loss',
color='orange', linewidth=2)

ax2.set_title('ANN Model Loss (Categorical Crossentropy)')

ax2.set_xlabel('Epochs')

ax2.set_ylabel('Loss')

ax2.legend()

ax2.grid(True)



plt.tight_layout()

plt.savefig("exp7_ann_learning_curves.png")

print("Saved 'exp7_ann_learning_curves.png'")



# 6. CLASSIFICATION REPORT & CONFUSION MATRIX

# Get predictions (these will be arrays of 22 probabilities)

y_pred_probs = model.predict(X_test_scaled)
```

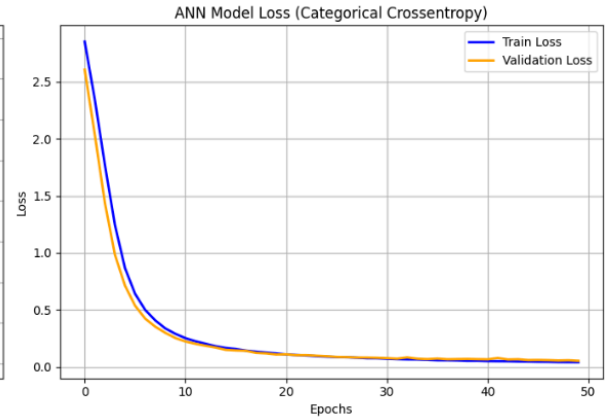
```
# Convert probabilities back to single class integers
y_pred = np.argmax(y_pred_probs, axis=1)

print("\n--- Classification Report ---")
# Convert numbers back to string labels for the report
target_names = label_encoder.inverse_transform(np.arange(22))
print(classification_report(y_test, y_pred, target_names=target_names))

print("\nGenerating Confusion Matrix...")
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='mako',
            xticklabels=target_names,
            yticklabels=target_names)

plt.title("ANN Confusion Matrix", fontsize=16)
plt.xlabel("Predicted Crop", fontsize=12)
plt.ylabel("True Crop", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig("exp7_ann_confusion_matrix.png")
```

```
print("Saved 'exp7_ann_confusion_matrix.png'. Experiment Complete!")
```

Output -







Conclusion -

The experiment successfully implemented an **Artificial Neural Network** that achieved high accuracy by learning non-linear environmental patterns for multi-class crop classification. Through careful **hyperparameter tuning** of layers and epochs, the model demonstrated robust convergence and effective mapping of complex soil and weather data.